



Détection d'anomalies dans les graphes temporels pour la cybersécurité

Rapport de stage de fin d'études

Alexandre Bouvier

5^e année Informatique et Statistique
Polytech Lille

Laboratoire d'accueil :

IRISA – Institut de Recherche en Informatique et Systèmes Aléatoires
Équipe MALT

Période du stage :

2 mars 2026 – 14 août 2026

Encadrants :

Elisa Fromont
Simon Malinowski
Barbara Pilastre

Encadrant école :

Thibault Liétard

Établissement :

Université de Rennes
IRISA

Août 2026

1 Remerciements

Je tiens à remercier chaleureusement mes tuteurs, Elisa Fromont, Simon Malinowski et Barbara Pilastre d'avoir rendu ce stage possible. J'ai grandement apprécié travailler sur ce sujet complexe et intéressant. Je retiens nos échanges, leur écoute et leur soutien tant sur le plan technique qu'humain. Je les remercie pour les opportunités proposées et leur confiance dans mon travail.

Je remercie également mon tuteur école, Thibault Liétard, pour nos échanges et ses conseils lors de sa visite en début de stage.

Ce stage marquant la fin de mon cursus à Polytech Lille, j'en profite pour remercier l'ensemble des enseignants de la formation IS. Malgré les critiques dont elle est parfois l'objet, j'ai personnellement pris un très grand plaisir à suivre les cours de cette formation que j'ai trouvée intellectuellement très enrichissante.

De même, je remercie tous les membres des équipes MALT et LACODAM pour leur sympathie, leurs conseils et le partage d'expérience. Je remercie en particulier l'assistante d'équipe, Gaëlle Tworkowski, pour l'accueil le premier jour et l'accompagnement au cours du stage.

Aussi, je souhaite mentionner mes deux collègues de bureau, Vincent Mari et Mathias Hebras, sans qui ce stage aurait été plus morne.

Enfin, comment terminer cette section sans remercier mon camarade de promo et ami, Elias Ramili, pour son soutien moral tout au long de cette période et son regard avisé sur mes présentations et rapports.

Sommaire

1 Remerciements	2
2 Introduction	4
3 Présentation	4
3.1 Laboratoire IRISA	4
3.2 Équipe MALT	4
4 Contexte	5
4.1 Motivation	5
4.2 Données	5
4.3 Objectifs	8
5 État de l’art et méthodologie	8
5.1 Étude bibliographique	8
5.2 Implémentation d’une pipeline	11
6 Expérimentations	12
6.1 Contraintes expérimentales	12
6.2 Premiers résultats	13
6.3 Représentation des données	14
6.4 Génération d’exemples négatifs	15
7 Valorisation des travaux	19
8 Discussion et perspectives	20
9 Bilan personnel	21
9.1 Environnement de recherche	21
9.2 Compétences techniques	21
9.3 Impact sur le projet professionnel	22
10 Conclusion	23
Références	24
A Annexes	26

2 Introduction

Dans le cadre de ma dernière année au sein de la formation Informatique et Statistiques à Polytech Lille, j'ai effectué un stage de recherche de 5 mois au sein du laboratoire IRISA de l'Université de Rennes. Ce stage, réalisé au sein de l'équipe MALT, s'est déroulé du 2 mars au 14 août 2026 à Rennes. Il est encadré par Elisa Fromont, professeure à l'Université de Rennes, Simon Malinowski, enseignant-chercheur à l'Université de Rennes, et Barbara Pilastre, Data scientist à l'agence ministérielle pour l'IA de la défense (AMIAD).

Le sujet du stage porte sur la détection d'anomalies dans les graphes temporels pour une application en cybersécurité. Il fait suite aux travaux menés par Florent Cheyron lors d'un stage en 2025. Le sujet sera l'objet d'une thèse débutant en automne 2026, dirigée par les encadrants de mon stage actuel.

Ce rapport a pour but de présenter le bilan des travaux réalisés durant l'ensemble du stage. Il décrit dans un premier temps le contexte et les enjeux du sujet. Ensuite, il détaille la méthodologie employée et les expérimentations réalisées. Enfin, il met en lumière les compétences acquises et les enseignements tirés de ce stage de recherche.

3 Présentation

3.1 Laboratoire IRISA

Créé au début des années 1970, l'IRISA ¹ (Institut de Recherche en Informatique et Systèmes Aléatoires) est une unité mixte de recherche (UMR) associant plusieurs établissements d'enseignement supérieur et organismes de recherche, notamment le CNRS, Inria, l'Université de Rennes, l'INSA Rennes et l'ENS Rennes. Le laboratoire est implanté sur les sites de Rennes, Lannion et Vannes.

L'IRISA est l'un des principaux laboratoires français en informatique. Il rassemble plus de quarante équipes de recherche réparties en plusieurs départements scientifiques, couvrant un large éventail de thématiques allant de l'intelligence artificielle et de la science des données à la cybersécurité, aux réseaux, à la robotique ou encore au calcul haute performance.

Mon stage s'est déroulé au sein de l'équipe MALT, rattachée au département Gestion des données et de la connaissance. Les recherches menées dans ce département portent sur la modélisation, la gestion et l'analyse de données complexes afin d'en extraire des connaissances dans des domaines variés.

3.2 Équipe MALT

L'équipe MALT ² (MACHINE Learning with Temporal constraints) est une équipe-projet commune à Inria et à l'IRISA spécialisée dans l'apprentissage automatique pour les données présentant une dimension temporelle. Ses travaux s'articulent autour de trois principaux axes de recherche.

Le premier axe concerne le développement de méthodes d'apprentissage pour les données temporelles, telles que les séries temporelles ou les graphes dynamiques. Les problématiques abordées incluent notamment l'apprentissage de représentations et la génération de données.

Le deuxième axe porte sur l'adaptation des modèles à l'évolution des données au cours du temps. Les recherches s'intéressent notamment à l'apprentissage continu et à l'adaptation de domaine.

Enfin, le troisième axe est consacré à la fiabilité des modèles d'apprentissage. Il regroupe des problématiques telles que l'explicabilité et la robustesse des méthodes développées.

1. <https://www.irisa.fr/>

2. <https://team.inria.fr/malt/>

4 Contexte

4.1 Motivation

Les cyberattaques représentent aujourd'hui une menace majeure pour les organisations de toutes tailles. Les cibles de ces attaques peuvent être des entreprises, administrations ou établissements de santé. Cela peut entraîner des interruptions de service ou des fuites de données. A titre d'exemple, en avril dernier, l'ANTS, l'agence délivrant les titres d'identité français, a été victime d'une cyberattaque exposant possiblement les données personnelles de plusieurs millions de comptes. De plus, les techniques employées par les attaquants évoluent rapidement et deviennent de plus en plus sophistiquées. L'émergence de modèles d'intelligence artificielle capables d'automatiser certaines étapes d'une attaque renforce encore cette menace. On peut notamment mentionner le modèle Claude Mythos de l'entreprise Anthropic, présenté comme capable d'identifier et d'exploiter des vulnérabilités cachées de systèmes de manière autonome.

Face à ces risques, les systèmes de détection d'intrusion (Intrusion Detection Systems, IDS) ont pour objectif de surveiller l'activité d'un système informatique afin d'identifier les comportements suspects le plus tôt possible. Ils s'appuient notamment sur l'analyse des logs, décrivant l'activité du système, et reposent de plus en plus sur des méthodes d'intelligence artificielle.

4.2 Données

4.2.1 Structure des données

La surveillance des systèmes informatiques nécessite l'exploitation des logs produits par le système. Dans notre cas, ces logs constituent notre seule source d'informations sur l'activité du système. Il est donc indispensable de comprendre leur structure et dépendance.

Les réseaux informatiques modernes produisent en continu une grande quantité de logs décrivant leur activité. Ces logs expriment des interactions entre des objets du système tels que des fichiers, des processus, etc. Un log pourrait donc être "le processus A lit le fichier B à l'instant t". Il est toujours structuré de la même manière (cf Figure 1). Il contient la date et l'heure de l'interaction, l'objet du système qui initie l'action (e.g., objet A), l'objet qui subit l'action (e.g., objet B), leur type (e.g., processus, fichier) et le type d'action réalisée (e.g., lecture). En outre, les logs incluent des informations supplémentaires décrivant les objets et l'interaction (e.g., la ligne de commande exécutée, le chemin d'accès). Les données brutes sont donc une séquence temporelle d'interactions entre objets du système.

Pris individuellement, un log ne contient souvent pas les informations suffisantes pour comprendre le comportement global du système. Il est donc nécessaire de considérer les dépendances entre les logs. La succession d'interactions entre objets, représentée par les logs, forme naturellement un graphe appelé graphe de provenance (cf Figure 2). Ce graphe capture l'historique de tous les événements se déroulant dans le système. Les nœuds du graphe correspondent aux objets du système, tandis que les arêtes représentent les interactions entre objets et sont orientées. Comme le système produit constamment de nouveaux logs, des arêtes et des nœuds apparaissent au cours du temps, ce qui rend le graphe dynamique. De plus, le graphe de provenance est hétérogène (les nœuds et arêtes sont de différents types) et multigraphe (plusieurs arêtes peuvent être liées plusieurs fois aux mêmes nœuds). Cette représentation des données sous forme de graphe temporel permet d'exposer des séquences d'actions et les dépendances relationnelles entre objets du système.

Parmi les séquences d'actions (arêtes) présentes dans le graphe de provenance, certaines sont susceptibles de correspondre à des attaques. Comme toutes autres activités du système, l'exécution d'une attaque sur le système produit des logs qui se matérialisent par un ensemble d'arêtes dans le graphe de provenance. Ces arêtes caractérisent des comportements malveillants et, à ce titre, ne devraient pas être observées dans le graphe de provenance d'un système en fonctionnement normal. Pour illustrer, la figure 2 montre

```

1  {
2      "action": "TERMINATE", \\
3      "actorID": "5689402e-9c41-4fe1-aff4-a0b43a8417cf", \\
4      "hostname": "SysClient0063.systemia.com", \\
5      "id": "15aff6c7-a663-4244-a97a-258fc66625ed",
6      "object": "PROCESS",
7      "objectID": "5689402e-9c41-4fe1-aff4-a0b43a8417cf",
8      "pid": 4772,
9      "ppid": 776,
10     "principal": "SYSTEMIACOM\\agilbard",
11     "properties": {
12         "acuity_level": "4",
13         "command_line": "\\??\\C:\\\\Windows\\system32\\conhost.exe",
14         "image_path": "\\Device\\HarddiskVolume1\\Windows\\conhost.exe",
15         "parent_image_path": "\\Device\\HarddiskVolume1\\Windows\\conhost.exe",
16         "sid": "S-1-5-18",
17         "user": "NT_AUTHORITY\\SYSTEM"
18     },
19     "tid": -1,
20     "timestamp": "2019-09-21T03:03:07.132-04:00"
21 }

```

FIGURE 1 – Exemple d'un log décrivant un processus se terminant de lui-même. "actorID" identifie le processus initiant l'action, "objectID" identifie l'objet subissant l'action, "action" décrit le type d'action réalisée et "timestamp" indique l'instant de l'interaction

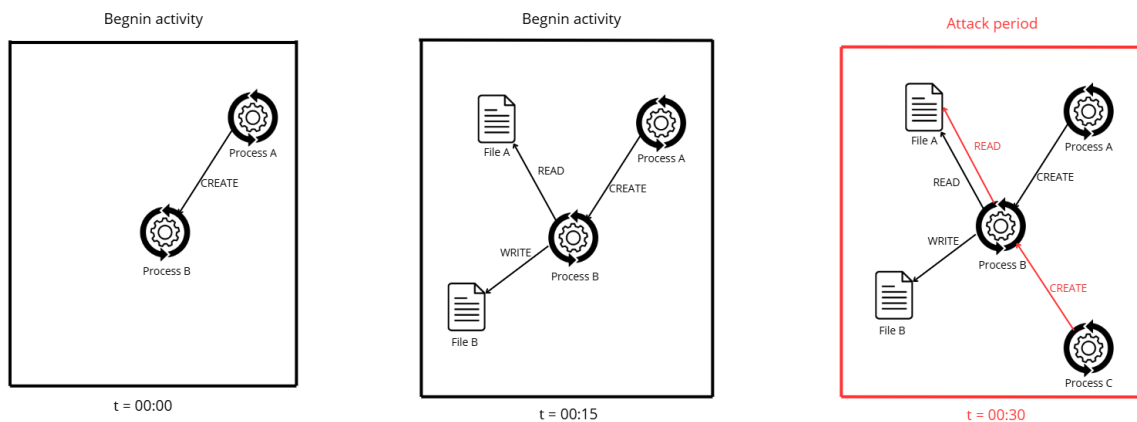


FIGURE 2 – Exemple d'un graphe de provenance

une attaque exécutant un processus qui lit un fichier sensible. On peut par exemple imaginer que le fichier est `etc/passwd` et permet ainsi à l'attaquant d'élever ses privilèges.

4.2.2 DARPA OpTC

Plusieurs jeux de données permettant d'évaluer des approches de détection d'intrusion existent. Nous utiliserons exclusivement le jeu de données DARPA OpTC (Operationally Transparent Cyber) fourni par la DARPA étant l'un des jeux de données les plus réalistes du domaine. En effet, DARPA OpTC est composé de plus de 17 milliards de logs collectés sur un système de 1000 machines. Il ne s'agit pas d'activité réelle, mais d'activité simulée sur une période de 9 jours. On peut diviser la période d'activité en 2 phases. Durant les 6 premiers jours, le système ne subit aucune attaque. Seule l'activité normale est enregistrée. Lors des 3 derniers jours, le système est attaqué par une Red Team. Trois types de cyberattaques sophistiquées sont employés sur trois jours différents. L'ensemble des cyberattaques représente 300 000 logs, labellisés comme malveillants dans le jeu de données, soit une proportion très faible de la totalité des logs enregistrés. Les données d'OpTC sont donc fortement déséquilibrées, ce qui rend le dataset réaliste par rapport à de vrais systèmes.

4.2.3 Challenges

Travailler sur des données de cybersécurité réalistes pour faire de la détection d'intrusion apporte de nombreuses contraintes et challenges justifiant notre choix d'approche et de méthodologie.

- **Une structure de données complexe**

Les graphes de provenance donnent une représentation complexe des données, demandant des méthodes capables de gérer les dépendances temporelles, relationnelles et structurelles. Elles doivent être capables d'apprendre les motifs du graphe à plusieurs échelles et tirer parti des dépendances temporelles à court et long terme. C'est pourquoi les méthodes d'apprentissage sur les graphes temporels semblent les plus appropriées.

- **Des grands volumes de données**

Une autre contrainte affectant le choix des méthodes employées est le grand volume de données à traiter. Cela implique d'utiliser des méthodes qui peuvent passer à l'échelle ou d'effectuer un premier tri dans les données à traiter.

- **Des attaques variées**

Les attaques peuvent également être très diverses. En outre, seule une petite fraction du large volume de données concerne des attaques, rendant le jeu de données fortement déséquilibré. Dès lors, il est impossible d'observer tous les types d'attaques existants dans les données d'entraînement. De plus, de nouveaux types d'attaques émergent régulièrement. Un modèle entraîné de manière supervisée sur des motifs d'attaques deviendrait inefficace pour détecter toutes les attaques non observées durant l'entraînement. Afin d'être capable de généraliser la détection à des types d'attaques inconnus, nous nous plaçons dans un cadre d'apprentissage semi-supervisé. Les données d'entraînement sont constituées uniquement d'activités normales du système. Une activité trop différente de l'activité normale est considérée comme une anomalie et donc une attaque.

- **Peu de jeux de données réalistes disponibles**

Enfin, pour des raisons de sécurité et de confidentialité, il n'existe qu'un faible nombre de jeux de données de cybersécurité réalistes. DARPA OpTC est l'un d'entre eux, ce qui justifie son utilisation.

L'analyse de la structure des données et des contraintes associées permet donc d'orienter le problème vers une approche de détection d'anomalies dans les graphes temporels.

4.3 Objectifs

L'objectif principal de ce stage est d'étudier à quel point les méthodes d'apprentissage sur les graphes temporels peuvent être exploitées pour améliorer la détection d'intrusions. On souhaite exploiter la richesse des graphes de provenance, représentant les interactions entre les objets du système et leur évolution temporelle, afin de modéliser le comportement normal d'un réseau et d'identifier les anomalies correspondant à des attaques.

Plusieurs pistes d'amélioration des modèles de prédiction de lien sont explorées. Une première option consiste à enrichir la représentation initiale des nœuds et des arêtes en exploitant les informations disponibles dans les logs, notamment les chemins d'accès et les lignes de commande. Une seconde piste d'amélioration consiste à améliorer l'étape de génération d'exemples négatifs lors de l'apprentissage des modèles de prédiction de liens, en explorant différentes stratégies permettant de produire des exemples plus réalistes.

Ce travail vise également à mieux comprendre les limites de ces approches. L'étude des différentes représentations des données et des stratégies de génération d'exemples négatifs doit permettre d'identifier les principales limitations de la détection d'anomalies via cette approche.

5 État de l'art et méthodologie

5.1 Étude bibliographique

5.1.1 Positionnement du sujet

Le problème de détection d'anomalies dans des graphes temporels est un domaine de recherche récent qui se situe à l'intersection de différentes thématiques. Pour comprendre le sujet, il est donc d'abord nécessaire de s'intéresser à des domaines tels que l'apprentissage dans les graphes, l'apprentissage dans les séries temporelles et la détection d'anomalies. C'est pourquoi j'ai mené en premier lieu une étude bibliographique qui est indispensable pour identifier les méthodes existantes et leurs limites.

Il existe une riche littérature sur des problématiques proches du sujet en question. On retrouve de nombreux travaux sur des méthodes de détection d'anomalies dans des séries temporelles, alors que d'autres s'intéressent à la détection d'anomalies dans les graphes statiques. En parallèle, les recherches traitant de l'apprentissage dans des graphes temporels se sont développées ces dernières années. Ces méthodes se focalisent généralement sur deux tâches principales, à savoir la classification de nœuds et la prédiction de liens. En revanche, à notre connaissance, un seul papier ([Reha et al., 2023](#)) traite spécifiquement de la détection d'anomalies dans les graphes temporels. Cette rareté peut s'expliquer par le fait que les méthodes d'apprentissage dans les graphes temporels sont récentes.

5.1.2 Principe de la prédiction de lien pour la détection d'anomalies

Ce constat motive donc notre approche. Elle consiste à employer des méthodes de prédiction de liens existantes et à les adapter pour résoudre une tâche de détection d'anomalies, plutôt que de développer une nouvelle architecture. L'hypothèse importante de cette approche est que si un modèle est capable d'apprendre le comportement normal d'un graphe temporel et de prédire les arêtes apparaissant dans le

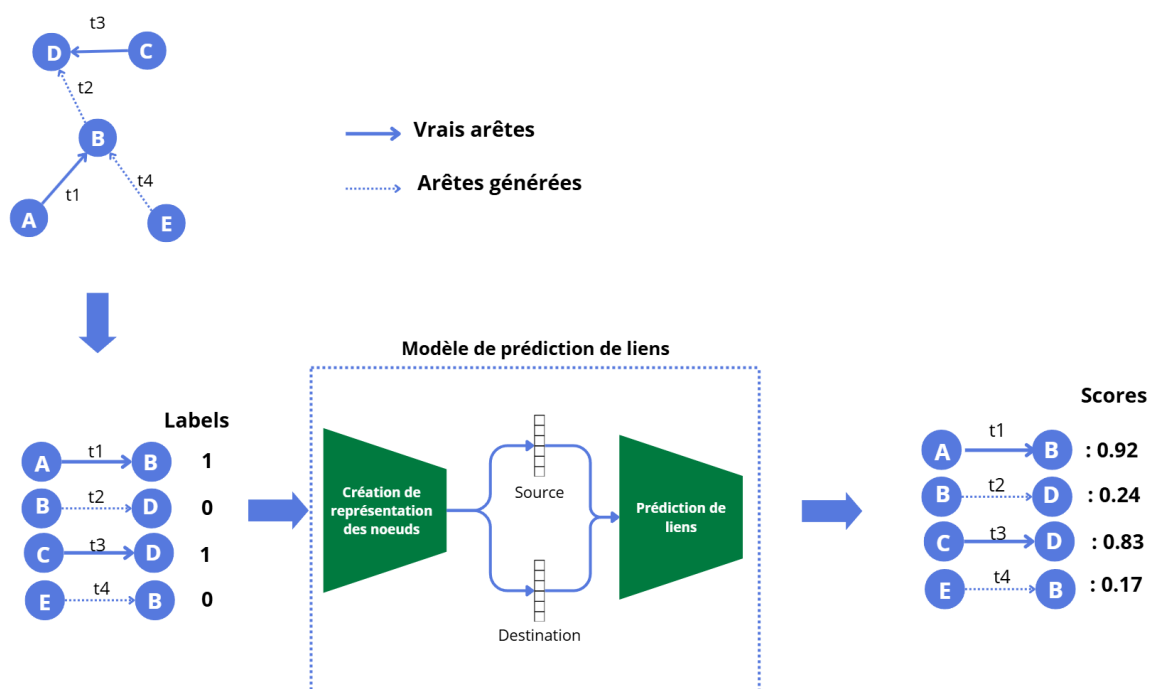


FIGURE 3 – Fonctionnement d'un modèle de prédiction de liens

futur. On peut alors considérer une anomalie comme étant une arête mal prédite, c'est-à-dire une arête existante du graphe et non prédite par le modèle. La détection d'anomalies devient alors un problème de prédiction de liens similaire à une tâche de classification binaire.

La prédiction de liens est une tâche qui vise à décrire l'évolution du graphe dans le futur. Plus précisément, le but est de déterminer si une arête est susceptible d'apparaître à un instant donné dans le futur, connaissant l'état actuel du graphe et ses états passés. Un modèle de prédiction de lien reçoit en entrée deux nœuds du graphe (e.g., A et B) et un instant t considéré. La sortie est un score entre 0 et 1. Un score proche de 1 indique qu'une arête a de grandes chances d'apparaître entre A et B à cet instant, tandis qu'un score proche de 0 suggère que l'arête est peu compatible avec le comportement habituel du graphe.

Pour entraîner un modèle à réaliser une tâche de prédiction de lien, on emploie la même stratégie que pour une tâche de classification binaire (cf Figure 3). Le modèle apprend à différencier les exemples positifs des exemples négatifs en minimisant une fonction de coût appelée cross-entropie. Dans notre cas, les exemples positifs (en trait plein sur la Figure 3) correspondent à des liens réellement observés dans le graphe. Les exemples négatifs (en traits pointillés sur la Figure 3), quant à eux, sont des arêtes n'existant pas dans les données et qui sont générées artificiellement. Afin d'assurer un jeu de données équilibré et de faciliter l'apprentissage, à chaque exemple positif correspond un exemple négatif. Il existe une multitude de façons de générer ces exemples négatifs, et la stratégie choisie peut avoir un impact très important sur ce que le modèle apprend, notamment en fonction de à quel point les exemples générés sont réalistes.

5.1.3 Les familles de méthodes de prédiction de liens

Une multitude de méthodes de prédiction de lien existent dans la littérature, mais elles suivent généralement le même principe. Dans un premier temps, elles créent une représentation de chaque nœud intégrant des informations spatiales et temporelles sur le nœud à un instant donné. Dans un second temps, le score final est déduit de ces représentations. Les différentes méthodes se différencient principalement par la manière dont elles construisent ces représentations.

Une première famille de méthodes repose sur les *Graph neural networks* (GNN) (cf Figure 4, schéma de

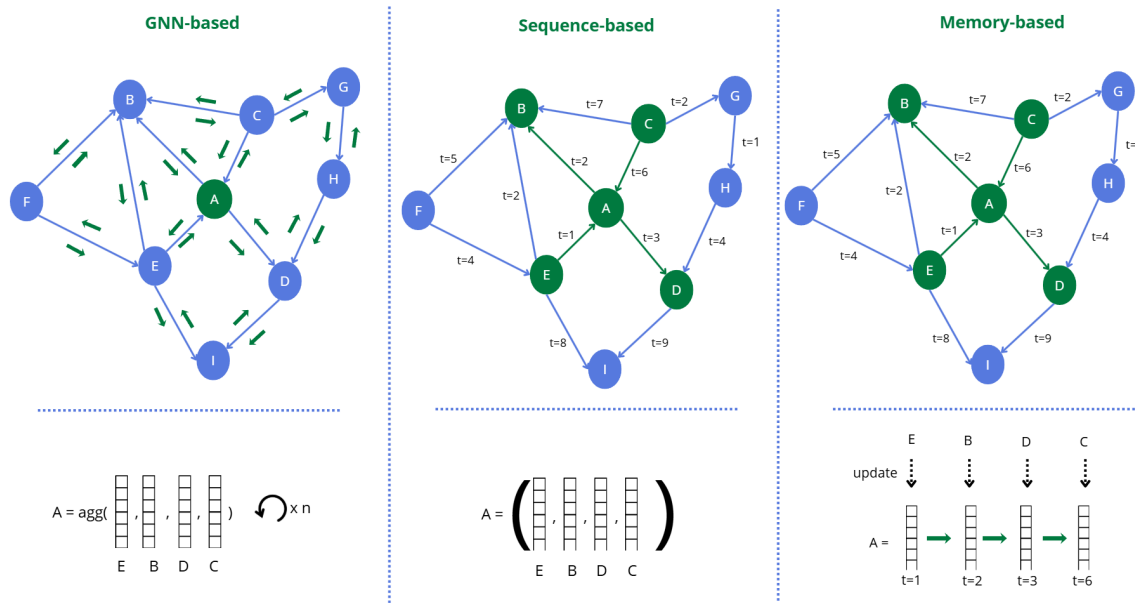


FIGURE 4 – Comparaison des méthodes de prédiction de liens

gauche). Il s'agit de l'architecture neuronale la plus connue pour l'apprentissage dans des graphes. Ces méthodes exploitent le principe de *message passing* selon lequel chaque nœud transmet des informations sur sa représentation à ses voisins. De la sorte, les nœuds enrichissent leur propre représentation. Après plusieurs itérations, le nœud crée une représentation qui résume la structure locale du graphe. Pour intégrer la dimension temporelle, certains travaux étendent l'architecture GNN en tirant parti des mécanismes d'attention présents dans les architectures Transformer. C'est par exemple le cas de TGAT (Xu et al., 2020), qui allie *message passing* et attention pour modéliser les dépendances d'arêtes successives.

D'autres méthodes n'utilisent pas le *message passing*. A la place, on considère directement l'historique des interactions de chaque nœud. Un nœud est représenté par la séquence formée par les représentations de ses voisins, ordonnés chronologiquement formant ainsi des séries temporelles multivariées. Ces séries temporelles servent ensuite de données d'entrée pour un réseau de neurones qui peut avoir différentes structures (cf Figure 4, schéma du centre). DyGFormer (Yu et al., 2023) traite les séquences avec une architecture transformer, tandis que GraphMixer (Cong et al., 2023) propose un encodage vectoriel du temps et un réseau de neurones multi-couches classique.

Enfin, une dernière famille de méthodes de prédiction de lien utilise des mécanismes similaires à ceux présents dans les architectures utilisées pour les séries temporelles. Chaque nœud est associé à un état de mémoire. Cet état est mis-à-jour après chaque nouvelle interaction impliquant le nœud en question (cf Figure 4, schéma de droite). Ainsi, les nœuds gardent une représentation condensée de leurs interactions passées, et la représentation évolue au fil du temps. La méthode TGN (Rossi et al., 2020) et TNCN (Zhang et al., 2024) utilisent ce principe pour modéliser le comportement dynamique des nœuds.

Certaines méthodes existantes ne rentrent dans aucune de ces familles. On peut citer par exemple Base3 (Emma, 2025), un modèle purement statistique ne nécessitant pas d'entraînement qui repose sur la popularité des nœuds, la fréquence d'apparition des arêtes et la co-occurrence de motifs temporels. TPNet (Lu et al., 2024) est un autre exemple de méthode exploitant les caractéristiques des chemins temporels existants entre les nœuds pour construire des représentations.

Ces différentes familles de modèles constituent aujourd'hui l'état de l'art en prédiction de liens dans les graphes temporels. Elles offrent des stratégies pour représenter l'évolution d'un graphe au cours du temps

et serviront de base aux expérimentations présentées dans la suite de ce rapport.

5.2 Implémentation d'une pipeline

5.2.1 Origine de la pipeline

Nous proposons un pipeline pour réaliser la détection d'anomalies sur les données OpTC en exploitant les méthodes de prédiction de liens présentées plus haut. Aussi, mon implémentation reprend des travaux précédents et repose sur une librairie python existante.

Les méthodes de prédiction de lien présentées plus haut sont issues de travaux variés. Pour pallier la diversité des implémentations, la librairie python DyGlib (Yu et al., 2023) unifie les méthodes d'apprentissage dans les graphes temporels dans une même pipeline. Elle prend en charge l'entraînement et l'évaluation des modèles pour deux tâches distinctes : la prédiction de lien et la classification de nœud. Cette librairie sert de socle à la pipeline finale présentée. Dans notre cas, nous nous intéressons uniquement à la tâche de prédiction de liens.

TGM est une autre librairie python implémentant des méthodes d'apprentissage sur les graphes temporels. Par rapport à DyGlib, cette librairie semble mieux maintenue et implémente des méthodes plus récentes. Nous avons un temps privilégié cette librairie pour l'implémentation d'une pipeline, mais elle a finalement été abandonnée au profit de DyGlib en raison de problèmes de passage à l'échelle rencontrés lors de l'implémentation de la librairie.

De plus, une première pipeline, basée sur la librairie Dyglib et intégrant un module de détection d'anomalies, avait été développée lors d'un précédent stage. Cette expérience a ainsi motivé le choix de mener les expérimentations de ce stage à l'aide de cette librairie.

Le prétraitement des données proposé dans la pipeline héritée du stage précédent ne permet pas de créer un encodage initial des nœuds et arêtes du graphe. Pour y remédier, on tire parti des travaux réalisés par Fanny Dijoub sur GRAAL (Dijoud et al., 2026), un système de détection d'intrusion basé sur la détection d'anomalies dans des graphes. La pipeline employée par GRAAL est appliquée aux données OpTC et propose un prétraitement des données incluant la possibilité d'encoder des représentations des nœuds et arêtes. J'ai donc intégré le prétraitement de GRAAL à la pipeline en veillant à conserver la cohérence entre les deux implémentations.

5.2.2 Description de la pipeline

Par la fusion des deux implémentations précédentes, nous obtenons une pipeline complète allant des données brutes à l'entraînement et l'évaluation des modèles (cf figure 5). On peut distinguer 3 étapes distinctes : le prétraitement, la tâche de prédiction de lien et la tâche de détection d'anomalies.

Les données brutes OpTC ne sont pas adaptées pour être traitées par les modèles de prédiction de lien et nécessitent donc d'être transformées. En effet, le jeu de données OpTC est composé d'une série de logs décrivant des interactions entre des objets du système. Les logs incluent également de nombreuses informations complémentaires. Les modèles implémentés par DyGlib ont besoin de trois informations en entrée : une liste des arêtes composées du nœud source, du nœud destination et de l'instant de l'interaction, les représentations initiales de chaque nœud et les représentations initiales de chaque arête. On transforme chaque log en une arête et chaque objet en un nœud. Les informations textuelles présentes dans les logs servent à initialiser les représentations des nœuds et des arêtes. Plusieurs encodages ont été testés et sont décrits dans une section suivante. Enfin, les données ont été découpées comme suit. Les données d'entraînement sont composées des 6 premiers jours des données qui ne contiennent pas d'attaque, moins les 12 dernières heures qui sont utilisées pour la validation de la tâche de prédiction de lien. Les trois derniers jours contenant des anomalies servent à évaluer le modèle sur une tâche de détection d'anomalies.

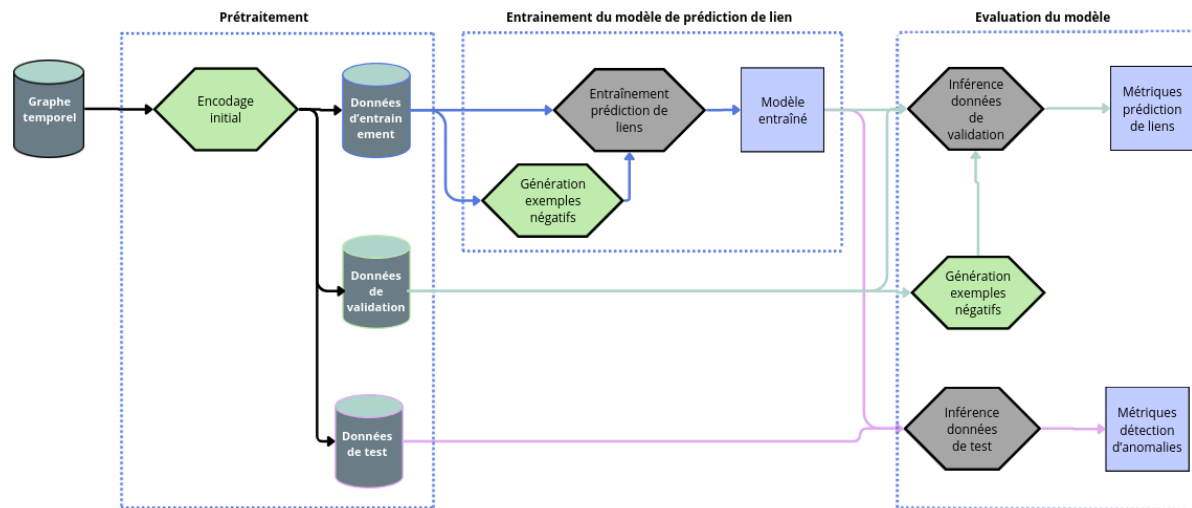


FIGURE 5 – Schéma de la pipeline complète

Une fois les données correctement prétraitées, nous entraînons un modèle de prédiction de lien à partir des données d'entraînement. Le but du modèle est d'apprendre le comportement normal des données. Pour ce faire, le modèle apprend à distinguer les exemples positifs des exemples négatifs qui lui sont fournis. Dans notre cas, les exemples positifs correspondent aux vraies arêtes présentes dans les données d'entraînement, tandis que les exemples négatifs sont des arêtes inexistantes qui sont générées artificiellement. L'impact des stratégies de génération est discuté dans l'une des sections qui suivent.

Nous évaluons le modèle entraîné sur deux tâches distinctes : la prédiction de lien et la détection d'anomalies. Les données de validation ne contiennent pas d'anomalies et la même stratégie de génération de faux exemples est exploitée pour déterminer les performances du modèle en prédiction de lien. Les résultats obtenus mesurent la capacité du modèle à résoudre la tâche sur laquelle il a été entraîné. Ceci est un pré-requis pour pouvoir résoudre la seconde tâche. Cette dernière est évaluée à partir des données contenant des anomalies. On mesure ici la capacité du modèle à séparer les données normales des anomalies qui devraient être considérées comme des exemples négatifs.

L'exploration de différentes approches et expérimentations a nécessité de nombreuses modifications d'implémentation à tous les niveaux de la pipeline. Ces expérimentations sont décrites dans la partie qui suit.

6 Expérimentations

6.1 Contraintes expérimentales

Le choix des expérimentations dépend non seulement des méthodes, mais aussi des ressources disponibles. Les données de cybersécurité manipulées sont volumineuses et les modèles de graphes temporels peuvent présenter un coût de calcul important. Nous avons donc dû adapter le protocole expérimental afin de rendre les expérimentations réalisables.

J'ai réalisé l'ensemble des expériences sur Grid5000, une plateforme française de calcul distribué pour la recherche scientifique. Grid5000 met à disposition un ensemble de clusters de calcul répartis sur plusieurs sites en France. Cela permet de réaliser des expériences nécessitant des ressources de calcul importantes, notamment l'entraînement de modèles de deep Learning sur GPU. Pour exécuter une expérimentation, je devais au préalable réserver un nœud de calcul pour une durée limitée.

Malgré ces ressources, les capacités de calcul et la mémoire disponible restent limitées. Avec plus de 17 milliards de logs sur un réseau de 1000 machines, le traitement complet n'est pas réalisable. Les

expérimentations ont donc été réalisées sur les données d’une seule machine du réseau, la machine n°051, représentant environ 5 millions de logs. Bien que cette réduction diminue le volume de données traité, elle permet de conserver un scénario réaliste tout en rendant les expérimentations possibles sur Grid5000

Les contraintes influencent également le choix des modèles. Les méthodes de prédiction de lien dans les graphes temporels présentent des besoins très variables en mémoire, en particulier lors de l’entraînement. Plusieurs architectures issues de la littérature n’ont ainsi pas pu être évaluées en raison de dépassements de mémoire.

Par conséquent, nos expérimentations se concentrent sur trois modèles différents : TGAT, fondé sur un mécanisme de message passing, DyGFormer et GraphMixer qui modélisent les interactions sous forme de séquences. Ces trois modèles permettent de faire varier les méthodes tout en restant compatibles avec les ressources disponibles sur Grid5000.

6.2 Premiers résultats

Nous avons réalisé une première série d’expérimentations pour établir une baseline sur le jeu de données OpTC. Les trois modèles retenus, TGAT, DyGFormer et GraphMixer, ont été entraînés sur les données de la machine n°051. Dans cette première étude, aucune représentation initiale des nœuds ou des arêtes n’est utilisée et les exemples négatifs sont générés selon la stratégie aléatoire proposée par défaut dans la librairie DyGLib.

Compte tenu du grand volume des données d’entraînement, les modèles sont entraînés pendant une unique époque. L’observation de la fonction de coût montre que celle-ci converge après une première passe sur les données. Une augmentation du nombre d’époques n’apporte donc pas d’amélioration significative.

Nous évaluons les modèles sur deux tâches distinctes. La première est la prédiction de lien, la tâche sur laquelle les modèles sont entraînés. Les performances sont mesurées à l’aide de l’accuracy (avec un seuil fixé à 0,5), de l’aire sous la courbe ROC (AUROC) et de l’aire sous la courbe précision-rappel (AUPRC). La seconde est la détection d’anomalies, évaluée à partir des mêmes métriques auxquelles s’ajoute le meilleur F1-score obtenu en faisant varier le seuil de décision.

Les premiers résultats (cf Table 1) montrent que les trois modèles obtiennent de bonnes performances en prédiction de lien. Ils sont capables de distinguer les arêtes observées des exemples négatifs générés artificiellement. En revanche, leurs performances en détection d’anomalies restent plus limitées.

TABLE 1 – Premiers résultats en prédiction de lien (AUROC) et en détection d’anomalies (F1-score, AUROC).

Modèle	Prédiction de liens	Détection d’anomalies	
	AUROC	F1	AUROC
GraphMixer	0.9592	0.0033	0.6480
DyGFormer	0.9556	0.0041	0.6415
TGAT	0.9518	0.0035	0.4171

Cette différence peut s’expliquer par le fort déséquilibre des données de test. Les interactions malveillantes représentent une proportion très faible de l’ensemble des arêtes observées. Dans ce contexte, même un petit nombre de faux positifs suffit à faire fortement diminuer la précision, et par conséquent, le F1-score. À l’inverse, l’AUROC est beaucoup moins sensible au déséquilibre et peut donc donner une vision trop optimiste des performances. Pour cette raison, le F1-score est une métrique plus représentative de la capacité réelle des modèles à détecter les attaques et sera privilégié.

Ces premiers résultats mettent en lumière un écart important entre les performances obtenues sur les deux tâches. Une première hypothèse est que les représentations apprises par les modèles ne sont pas assez discriminantes pour distinguer les interactions normales des interactions malveillantes. Une première piste d'amélioration consiste donc à enrichir les représentations initiales des nœuds et des arêtes. Cette hypothèse est explorée dans la section suivante.

6.3 Représentation des données

Pour apprendre le comportement normal d'un graphe temporel, les modèles de prédiction de liens exploitent trois sources d'information : la topologie du graphe, la dimension temporelle et les représentations initiales des nœuds et arêtes. Ces dernières caractérisent les nœuds et arêtes et permettent de différencier des objets ayant des comportements différents malgré une position similaire dans le graphe. C'est pourquoi l'une des principales pistes explorées cherche à enrichir cette représentation initiale.

En l'absence d'encodage initial, Le modèle de prédiction considère que tous les nœuds sont a priori identiques, de même pour les arêtes. Pour construire les représentations temporelles propres à chaque nœud, le modèle peut uniquement s'appuyer sur la structure locale du graphe et de la chronologie des interactions. Étant donné qu'un graphe temporel en lui-même est déjà très riche en informations, on peut imaginer qu'il contient suffisamment d'informations pertinentes pour apprendre le comportement du graphe. De ce fait, Nous utilisons cette première approche comme première baseline permettant d'évaluer l'apport des informations supplémentaires intégrées dans les représentations initiales.

Notre première étape a consisté à déterminer quelles sont les informations les plus pertinentes à intégrer dans la représentation. Les logs contiennent de nombreux attributs décrivant les interactions observées et les objets du système. On retrouve notamment des identifiants uniques tels que le PID des processus ou le SID des utilisateurs. Ces informations apportent peu de connaissances sur leur comportement. De manière plus intéressante, des données textuelles sont présentes parmi lesquelles les chemins d'accès des fichiers et processus, les chemins d'accès des processus parents ou encore les lignes de commandes. Ces informations décrivent la nature des entités manipulées et des actions réalisées. Par la suite, nous avons décidé de représenter chaque nœud par son chemin d'accès, tandis que chaque arête est représentée par la ligne de commande exécutée lorsque l'information est disponible.

6.3.1 Représentation vectorielle de données textuelles

Les modèles de prédiction manipulent des représentations vectorielles. Cette contrainte implique donc d'être en mesure de convertir les informations textuelles souhaitées en vecteurs numériques avant d'être exploitées. Cette problématique a été largement étudiée dans la littérature dans le domaine de la modélisation du langage et de la représentation vectorielle des mots. Le but est de représenter un mot ou une séquence de mots par un vecteur dense qui représente son contenu sémantique, de sorte que des mots de sens proche aient des vecteurs similaires. Cette représentation vectorielle peut être apprise en utilisant les données d'entraînement, puis appliquée à l'ensemble des données. En pratique, les informations textuelles que nous cherchons à encoder ont besoin de subir un prétraitement spécifique pour être apprises efficacement par le modèle. En effet, les chemins d'accès et lignes de commande ne sont pas rédigés en langage naturel. Nous les avons donc découpés selon des séparateurs comme "/" et nettoyés de certains symboles non informatifs.

Parmi les nombreux modèles de représentation vectorielle des mots, Word2Vec ([Mikolov et al., 2013](#)) est l'un des modèles les plus simples et les plus largement utilisés. Il s'agit d'un réseau de neurones peu profond dont le but est d'apprendre la relation entre un mot et son contexte. Le modèle Word2Vec apprend à prédire un mot en fonction de son contexte en faisant l'hypothèse que deux mots souvent placés dans un même contexte ont un sens proche. Pour ce faire, le modèle apprend une représentation vectorielle latente des mots. Nous utilisons cette représentation latente comme encodage. Pour les chemins

d'accès et les lignes de commande, nous agrégeons les vecteurs des mots qui les composent, par une moyenne dans notre cas. Nos expérimentations menées en employant ce modèle pour la représentation des données montrent que Word2Vec n'apporte pas d'informations pertinentes pour la prédiction de liens et la détection d'anomalies (cf Table 2). On peut alors supposer que Word2Vec est un modèle trop simple pour capturer la complexité des chemins d'accès et lignes de commande.

Le domaine de la modélisation du langage s'est grandement développé avec l'apparition des modèles pré-entraînés, aussi appelé modèle de fondation. Contrairement à Word2Vec, Ces nouvelles architectures permettent de construire des représentations contextualisées d'une phrase ou d'une séquence et pas uniquement d'un seul mot. BERT (Bidirectional Encoder Representations from Transformers) (Devlin et al., 2019) est un exemple de modèle de langage très utilisé basé sur l'architecture Transformer. Pré-entraîné sur un très grand corpus de texte, les représentations créées par BERT sont très riches. De plus, en raison du pré-entraînement, il fournit directement des représentations pour les chemins d'accès et les lignes de commande sans avoir besoin d'être entraîné au préalable. Son utilisation apporte une amélioration nette pour l'ensemble des modèles sur la tâche de prédiction de liens et de détection d'anomalies. Les performances en détection d'anomalies restent néanmoins très faibles (cf Table 2).

Le modèle pré-entraîné BERT peut être très expressif de par la richesse du corpus sur lequel il a été entraîné, mais il reste un modèle généraliste. Or, les logs de cybersécurité possèdent un vocabulaire et une structure non présents dans les textes classiques. C'est pourquoi nous avons procédé à une étape de fine-tuning sur nos données afin d'adapter les représentations produites à ce domaine. Cette approche est motivée par des travaux précédents (Guan et al., 2024) exploitant le fine-tuning d'un modèle BERT pour la représentation des logs. Cette phase permet de conserver les connaissances généralistes du modèle tout en le rendant spécialiste de la représentation de logs grâce à l'entraînement d'une faible quantité de données. En termes de performances, le fine-tuning du modèle ne semble rien apporter (cf Table 2). Cela peut signifier que le modèle BERT pré-entraîné était déjà suffisamment performant sur les données de logs.

6.3.2 Ajout des types de nœuds et d'arêtes

Enfin, les informations textuelles ne constituent pas la seule source d'information disponible. Les logs décrivent également la nature des objets manipulés et des interactions observées. Nous avons donc étudié l'intérêt d'intégrer explicitement le type des nœuds (processus, fichier, etc.) ainsi que le type des arêtes au sein des représentations initiales, afin d'évaluer l'apport de ces informations structurelles aux performances des modèles. L'ajout des types de nœuds et d'arêtes n'améliore pas les résultats précédents (cf Table 2). Pour l'expliquer, on peut émettre l'hypothèse que l'information du type est déjà contenue dans les chemins d'accès et lignes de commande.

L'ajout d'une représentation initiale des nœuds et arêtes utilisant les informations présentes dans les logs améliore la capacité du modèle de prédiction de lien à distinguer les vraies arêtes de celles créées artificiellement (l'ensemble des résultats est disponible en annexe, Figure 4). Cependant, Ces meilleures capacités ne se sont pas traduites par une grande amélioration des performances en détection d'anomalies. De par ces résultats, il est clair que la représentation initiale n'est pas le principal facteur limitant. Nous nous intéressons donc, dans la section suivante, à un autre élément de l'apprentissage : la génération des exemples négatifs.

6.4 Génération d'exemples négatifs

Le choix de la stratégie de génération d'exemples négatifs constitue l'un des principaux leviers d'amélioration des modèles de prédiction de lien. Les modèles apprennent à distinguer les interactions observées dans le graphe des interactions qui ne devraient pas exister. Puisque les exemples négatifs sont des arêtes qui n'existent pas naturellement dans le graphe, ils doivent être générés artificiellement. Il existe une

TABLE 2 – Performances des différents encodages en prédiction de lien (AUROC) et en détection d'anomalies (F1-score, AUROC).

Encodage	Modèle	Prédiction de liens	Détection d'anomalies	
		AUROC	F1	AUROC
Sans features	GraphMixer	0.9592	0.0033	0.6480
	DyGFormer	0.9556	0.0041	0.6415
	TGAT	0.9518	0.0035	0.4171
Word2Vec	GraphMixer	0.9594	0.0031	0.6474
	DyGFormer	0.9539	0.0034	0.6081
	TGAT	0.9471	0.0024	0.5202
BERT pré-entraîné	GraphMixer	<u>0.9903</u>	<u>0.0061</u>	0.7304
	DyGFormer	0.9837	0.0051	0.6916
	TGAT	0.9706	0.0073	0.6284
BERT fine-tuné	GraphMixer	0.9911	0.0052	0.7025
	DyGFormer	0.9665	0.0060	0.6645
	TGAT	0.9725	0.0047	0.7060
BERT + type	GraphMixer	0.9871	0.0043	<u>0.7181</u>
	DyGFormer	0.9887	0.0029	0.4836
	TGAT	0.9686	0.0057	0.6268

multitude de stratégies pour générer des exemples négatifs. Le réalisme de ces derniers influence directement ce que le modèle apprend. Des exemples trop peu réalistes conduisent le modèle à résoudre une tâche artificiellement simple. A l'inverse, des exemples plus réalistes permettent au modèle d'apprendre une représentation fidèle du comportement normal du système et donc de mieux détecter les anomalies.

Par défaut, les modèles de prédiction de lien implémentent une génération d'exemples négatifs aléatoires. Pour chaque arête réellement observée, on remplace le nœud de destination par un nœud choisi uniformément parmi l'ensemble des nœuds du graphe. On obtient une nouvelle arête n'existant pas dans le graphe original. Elle peut donc servir d'exemple négatif. Cette génération aléatoire présente l'inconvénient de générer des fausses arêtes très faciles à distinguer des vraies. Les nœuds sélectionnés pour former les arêtes peuvent par exemple être très éloignés dans le graphe ou dans leur évolution temporelle, les rendant donc peu réalistes. Il en résulte que le modèle apprend une frontière de décision peu pertinente, nuisant à la généralisation. En particulier, les attaques que l'on cherche à détecter via le modèle de prédiction ne forment pas des interactions arbitraires mais restent cohérentes tout en s'éloignant du comportement habituel du système.

L'objectif des expérimentations menées est de générer des exemples négatifs plus proches de la distribution des vraies données. Nous avons exploré plusieurs stratégies de génération d'exemples plus réalistes. Premièrement, nous nous sommes basés sur certaines heuristiques pour choisir des nœuds non pas parmi tous les nœuds existants, mais uniquement parmi les nœuds les plus réalistes. Une autre approche explorée consiste à créer directement de fausses représentations temporelles de nœuds via un modèle de diffusion, plutôt que de choisir des nœuds existants. Enfin, nous avons cherché à générer un ensemble de fausses arêtes cohérentes entre elles pour simuler des comportements plus réalistes. Toutes poursuivent le même objectif : rendre les exemples négatifs suffisamment réalistes pour améliorer la qualité des représentations apprises par le modèle.

6.4.1 Heuristiques

Une alternative à la génération d'exemples aléatoires présente dans la littérature exploite l'historique du graphe. Pour une arête existante donnée, plutôt que de remplacer le nœud destination par un nœud aléatoirement choisi dans tout le graphe, il est sélectionné parmi les nœuds connectés au nœud source par le passé. Expliqué autrement, nous réutilisons des arêtes observées à un instant précédent comme de fausses arêtes à un instant ultérieur. L'idée sous-jacente est qu'une arête ayant existé par le passé est un exemple plus plausible qu'une arête entièrement aléatoire. L'utilisation de cette méthode ne semble pas apporter de gains significatifs en termes de détection d'anomalies (cf Table 3). Cette heuristique n'est donc sans doute pas pertinente dans notre contexte.

Nous avons testé plusieurs autres heuristiques visant à privilégier des interactions plausibles. La première repose sur la popularité des nœuds. Plus un nœud a été lié à d'autres nœuds par le passé, plus la probabilité d'être choisi lors de la génération d'une fausse arête est élevée. Favoriser les nœuds populaires permet de créer des exemples négatifs impliquant des entités fréquemment observées. La seconde approche repose sur la proximité topologique. Pour créer un exemple négatif, le nœud destination est choisi parmi les voisins des voisins du nœud source. On émet ainsi l'hypothèse que deux nœuds proches ont plus de chances d'interagir que deux nœuds éloignés. Les deux approches donnent des résultats drastiquement différents. L'approche basée sur la popularité produit des résultats moins bons que la génération aléatoire en détection d'anomalies (cf Table 3), indiquant que cette heuristique n'est pas adaptée et fausse complètement l'apprentissage du comportement normal du modèle. À l'inverse, l'approche basée sur la proximité semble grandement améliorer les performances en détection d'anomalies sur l'ensemble des modèles testés. Cela renforce donc l'hypothèse que deux nœuds proches sont plus susceptibles d'interagir que deux nœuds éloignés.

Les différentes approches heuristiques ont certaines limites. Selon la structure des données, une stratégie peut être adaptée dans un contexte mais générer des exemples irréalistes dans un autre. À l'inverse, elle peut également générer des arêtes trop réalistes, indiscernables des vraies arêtes. Afin de dépasser ces limites, une approche générative a été explorée.

6.4.2 Modèle de diffusion

La stratégie de génération d'exemples avec un modèle de diffusion permet de générer directement de nouvelles représentations temporelles plutôt que de les construire à partir de nœuds existants. Les exemples générés ne correspondent pas nécessairement à des nœuds existants mais à des représentations synthétiques dont on peut contrôler le degré de réalisme. Cette approche est inspirée de travaux proposant ce type de stratégie pour les graphes statiques (Nguyen et Fang, 2024). J'ai adapté l'implémentation pour un usage sur les graphes temporels.

Le modèle de diffusion est entraîné en même temps que le modèle de prédiction de liens. Sachant la représentation temporelle d'un nœud produite par le modèle de prédiction de liens, le modèle de diffusion apprend la distribution des représentations de ses voisins. Pour apprendre cette distribution, la représentation des nœuds est progressivement bruitée, puis le modèle apprend à la débruiter. Il est ensuite capable de générer la représentation d'un voisin, connaissant la représentation du nœud. En choisissant à quel point la représentation générée est débruitée, nous pouvons créer des voisins plus ou moins proches de la vraie distribution (cf Figure 6). Les expérimentations sur le modèle de diffusion n'ont pas abouti. En effet, le modèle n'arrive pas à apprendre la distribution des représentations. On peut imaginer plusieurs causes possibles. La première est que les nœuds n'ont pas assez de voisins pour pouvoir apprendre une distribution. Le modèle de diffusion apprend la distribution à partir de 20 voisins. Si le nombre de voisins est inférieur, le modèle les duplique. Or, la majorité des nœuds dans notre graphe de provenance ont entre un et trois voisins, ce qui peut être insuffisant pour apprendre la distribution. La seconde cause possible est que les nœuds ont des représentations trop variées. Cette approche ayant été testée en fin de stage, je n'ai pas eu le temps de poursuivre cette piste davantage.

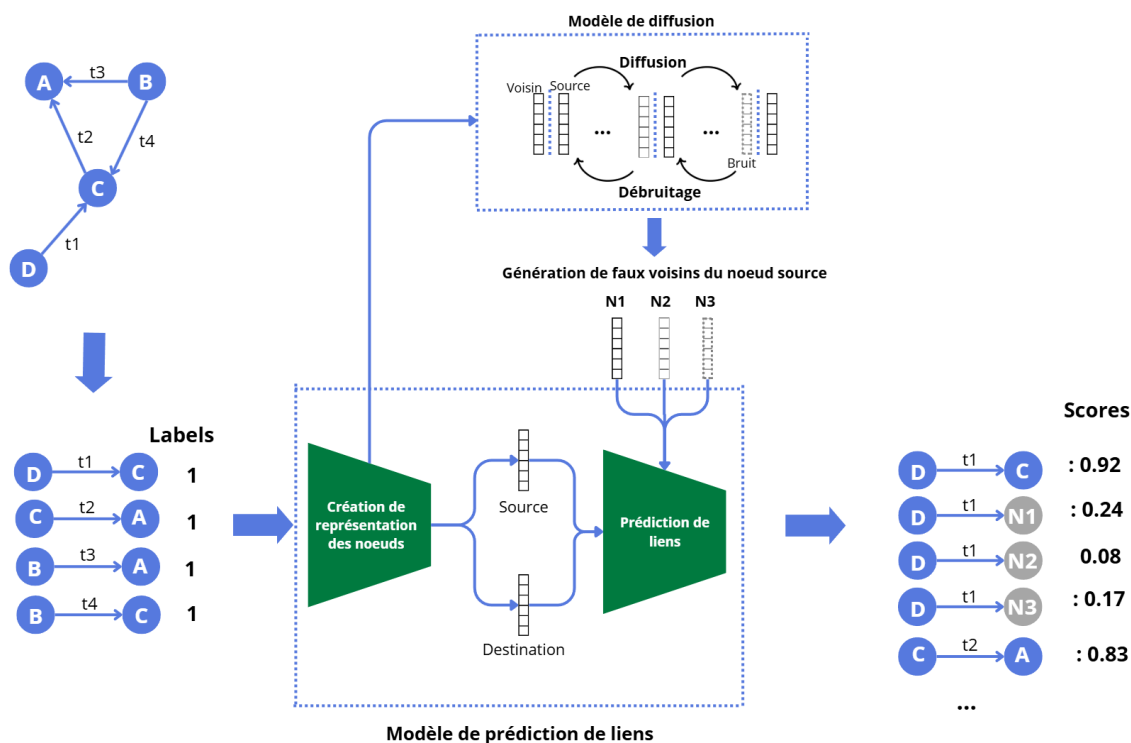


FIGURE 6 – Fonctionnement du modèle de diffusion

6.4.3 Motifs temporels

Les approches précédentes génèrent des arêtes indépendantes les unes des autres. Cependant, une attaque informatique ne correspond pas à une interaction unique, mais à une succession d'actions cohérentes formée de plusieurs arêtes. Nous avons donc voulu savoir s'il était plus simple pour le modèle de détecter une attaque si nous générions des séquences d'arêtes négatives plutôt que des arêtes négatives indépendantes les unes des autres. Suivant cette hypothèse, nous souhaitons générer des motifs temporels regroupant plusieurs arêtes comme exemples négatifs plutôt que des arêtes individuelles.

La génération de motifs temporels est un problème complexe qui nécessite de créer un ensemble d'arêtes différentes de celles présentes dans le graphe, mais suffisamment réaliste malgré tout. Nous ne traiterons pas ce problème. A la place, on se demande si cette approche est intéressante en se plaçant dans le meilleur cas possible, c'est-à-dire si les exemples générés avaient la même distribution que les anomalies que l'on cherche à détecter. Pour ce faire, nous extrayons des motifs temporels des anomalies présentes dans les données de test et nous les insérons dans les données d'entraînement en tant qu'exemples négatifs. nous avons testé deux variantes : l'une utilise des motifs temporels de taille 2, tandis que l'autre considère l'ensemble des anomalies présentes dans les données de test comme un unique motif temporel. Étant donné que le modèle observe les mêmes anomalies dans les données d'entraînement et dans les données de test, on s'attend à ce que de bonnes performances en prédiction de lien se traduisent par de bonnes performances en détection d'anomalies. Cependant, on observe qu'il est toujours difficile pour le modèle de détecter les anomalies malgré de bonnes capacités de prédiction de liens (cf Table 3). Ces résultats sont difficiles à expliquer. Notre principale hypothèse est qu'en extrayant des motifs temporels des attaques, on perd une partie de l'information liée au contexte dans lequel elles sont présentes (i.e., les arêtes bénignes existant autour de l'attaque). Or, la représentation apprise par le modèle dépend fortement de ce contexte. Il faudrait davantage d'investigation pour trouver la cause exacte du phénomène.

De par l'ensemble de ces expérimentations sur la génération d'exemples négatifs, il est clair que le choix de stratégie impacte grandement l'apprentissage du modèle (l'ensemble des résultats est disponible en annexe, Figure 5). L'utilisation d'heuristiques simples ne permet pas systématiquement d'améliorer les

TABLE 3 – Performances des différentes stratégies de génération d'exemples négatifs en prédiction de lien (AUROC) et en détection d'anomalies (F1-score, AUROC).

Negative sampling	Modèle	Prédiction de liens	Détection d'anomalies	
		AUROC	F1	AUROC
Aléatoire	GraphMixer	0.9871	0.0043	0.7181
	DyGFormer	0.9887	0.0029	0.6268
	TGAT	0.9686	0.0057	0.4836
Historique	GraphMixer	0.9853	0.0058	0.5652
	DyGFormer	0.9461	0.0030	0.5563
	TGAT	0.9884	0.0047	0.5171
Popularité	GraphMixer	0.9754	0.0022	0.3131
	DyGFormer	0.9836	0.0023	0.2920
	TGAT	0.9684	0.0025	0.2693
Voisins de voisins	GraphMixer	0.9932	0.0059	0.7628
	DyGFormer	0.9949	0.0063	0.7902
	TGAT	0.9894	0.0099	0.7622
Motifs taille 2	GraphMixer	0.9962	0.0063	0.7000
	DyGFormer	0.9901	0.0132	0.6578
	TGAT	0.9675	0.0022	0.5939
Motif global	GraphMixer	0.9996	<u>0.0112</u>	0.8538
	DyGFormer	0.9994	0.0102	<u>0.8353</u>
	TGAT	<u>0.9995</u>	0.0078	0.8027

résultats et dépend fortement du contexte. Dans notre cas, la génération de motifs en exploitant les voisins de voisins semble être celle créant les exemples les plus réalistes, ce qui encourage à employer des heuristiques reposant sur la proximité. La génération d'arête par un modèle de diffusion n'a pas pu être testée et peut être une piste d'amélioration. Enfin, la génération de motifs temporels comme exemples négatifs est un problème complexe. Pour l'heure, la principale limitation est la création d'exemples négatifs suffisamment représentatifs des anomalies à détecter.

7 Valorisation des travaux

Au cours de ce stage, j'ai eu l'opportunité de présenter mes travaux à deux reprises dans le cadre de workshops scientifiques. Un workshop est une journée d'échanges réunissant des chercheurs autour d'une thématique. Il permet de présenter des travaux en cours et de partager des résultats préliminaires.

La première présentation a eu lieu en avril lors d'un workshop sur le thème des séries temporelles, organisé par l'équipe MALT à l'occasion de la visite de Geoff Webb, professeur à l'Université Monash (Australie). Plusieurs membres de l'équipe, doctorants et chercheurs, ont présenté leurs travaux. J'y ai coanimé une présentation avec ma tutrice, Barbara Pilastre, portant sur le sujet du stage. Nous y avons présenté notre approche ainsi que les premiers résultats obtenus sur les représentations initiales des nœuds et des arêtes.

La seconde présentation s'est déroulée en juin lors d'un workshop consacré à l'IA et la cybersécurité, organisé par le cluster SEQUOIA à la suite des PEPR IA Days. Le cluster SEQUOIA rassemble des acteurs académiques travaillant sur les thématiques de l'intelligence artificielle et de la cybersécurité. Les PEPR IA Days est une rencontre nationale dédiée aux projets de recherche soutenus par le programme

PEPR en intelligence artificielle. Ce workshop réunissait des chercheurs, doctorants et industriels venus présenter leurs travaux sur l'application de l'IA à la cybersécurité.

À cette occasion, mes encadrants étant indisponibles, j'ai réalisé seul une présentation de vingt minutes, suivie de dix minutes d'échanges avec l'audience. J'y ai présenté le contexte du sujet, la méthodologie développée ainsi que les résultats obtenus sur les expérimentations concernant les représentations des données et les stratégies de génération d'exemples négatifs. Les deux présentations ont été réalisées en anglais.

Ces présentations ont constitué un bon exercice de communication scientifique et également une opportunité d'obtenir quelques retours de chercheurs travaillant sur des problématiques proches.

8 Discussion et perspectives

La détection d'intrusion à partir de données de cybersécurité réalistes constitue un problème difficile. Les données sont volumineuses et présentent une structure complexe mélangeant des dépendances temporelles et relationnelles. Les attaques ne représentent qu'une petite fraction des interactions observées et peuvent être très variées. Cela rend difficile l'apprentissage des comportements malveillants. Dans ce contexte, nous avons choisi d'aborder le problème comme une tâche de détection d'anomalies et d'exploiter des méthodes de prédiction de lien dans les graphes temporels pour modéliser le comportement normal du système.

Une première série d'expérimentations a consisté à enrichir la représentation initiale des nœuds et des arêtes en exploitant les informations textuelles contenues dans les logs. Différents encodages fondés sur des modèles de langage tels que Word2Vec et BERT ont été étudiés. Ces encodages, notamment ceux utilisant le modèle BERT, montrent que ces informations améliorent les performances des modèles sur leur tâche d'apprentissage, à savoir la prédiction de lien. Les chemins d'accès et les lignes de commande apportent donc une information utile permettant de mieux caractériser les interactions du système.

Cependant, cette amélioration ne se traduit pas par une augmentation significative des performances en détection d'anomalies. Ce résultat suggère que la qualité des représentations initiales ne constitue pas la principale limitation de notre approche. Plus généralement, nos expérimentations montrent qu'un modèle performant en prédiction de lien n'est pas nécessairement performant pour détecter des attaques. Cette observation met en lumière un décalage entre la tâche d'apprentissage utilisée et l'objectif final de détection d'intrusion.

À partir de ce constat, nous avons donc cherché à agir directement sur le processus d'apprentissage en modifiant la stratégie de génération des exemples négatifs. Les différentes heuristiques explorées montrent que le réalisme des exemples négatifs influence fortement les performances en détection d'anomalies. En particulier, les stratégies reposant sur la proximité topologique des nœuds produisent des exemples plus difficiles à distinguer des interactions normales et conduisent aux meilleurs résultats parmi les approches heuristiques évaluées.

Nous avons également exploré une approche consistant à générer des motifs temporels plutôt que des arêtes indépendantes. Cette direction paraît particulièrement prometteuse. En effet, une cyberattaque ne correspond généralement pas à une interaction isolée mais à une succession cohérente d'événements. Générer des ensembles d'interactions réalistes semble donc plus pertinent que de considérer chaque arête indépendamment. Les premiers résultats obtenus sont encourageants, mais démontrent surtout la difficulté à générer des motifs représentatifs des attaques. De nombreuses questions restent ouvertes concernant la manière de construire automatiquement des motifs suffisamment réalistes, variés et représentatifs du comportement normal.

Une autre limite de notre approche concerne le niveau auquel sont détectées les anomalies. Dans ce travail, la détection est réalisée au niveau des arêtes. Or, dans un contexte opérationnel, on considère

généralement qu'une attaque est correctement détectée si au moins une anomalie est détectée durant la période de l'attaque. Une telle approche peut ainsi constituer une piste d'amélioration, pour atténuer le déséquilibre entre données normales et anomalies.

Enfin, bien que ce travail se soit concentré sur les méthodes de prédiction de lien dans les graphes temporels, d'autres approches mériteraient d'être étudiées. Une première possibilité consiste à exploiter les nombreuses méthodes de détection d'anomalies développées pour les séries temporelles en faisant abstraction de la structure graphique des données. À l'inverse, il serait également envisageable d'utiliser des méthodes de détection d'anomalies dans les graphes statiques en négligeant la dimension temporelle.

9 Bilan personnel

9.1 Environnement de recherche

Ce stage constitue ma première expérience dans un laboratoire de recherche. L'IRISA rassemble un grand nombre d'équipes travaillant sur des problématiques très variées. J'ai également été surpris par la diversité des travaux menés au sein même de l'équipe MALT. Les travaux peuvent être théoriques ou plus applicatifs et portent sur différentes problématiques comme les séries temporelles, la génération de données ou l'apprentissage continu.

J'ai également découvert une partie du fonctionnement du processus de publication scientifique. La lecture d'articles m'a appris à identifier les principales conférences du domaine et à prendre en compte leur niveau de reconnaissance. J'ai également pu observer le fonctionnement de la recherche en pratique à travers les échanges avec les chercheurs, les doctorants de l'équipe et mes encadrants. Enfin, les deux workshops auxquels j'ai participé m'ont permis d'élargir ces échanges en étant confronté à des questions et des points de vue que je n'avais pas nécessairement envisagés.

9.2 Compétences techniques

9.2.1 Traiter des données complexes

Le traitement des données a été une part importante de mon travail. Les données OpTC sont particulièrement complexes. Il a donc été nécessaire de comprendre précisément la structure des données et de définir une représentation adaptée au problème étudié.

J'ai notamment travaillé sur la transformation des logs en graphes de provenance temporels, en identifiant les objets comme des nœuds et les interactions comme des arêtes. J'ai également extrait et prétraité les informations textuelles présentes dans les logs, notamment les chemins d'accès et les lignes de commande, afin de les transformer en représentations vectorielles.

Ce travail m'a permis de mieux comprendre l'importance de la phase de prétraitement des données. Avant d'entraîner un modèle, il est nécessaire de comprendre ce que représentent les observations et quelles informations doivent être conservées dans leur représentation. Dans notre cas, la structure temporelle et relationnelle des données a déterminé le choix des méthodes utilisées.

9.2.2 Développer et interpréter des modèles

Le stage m'a également permis de mettre en pratique différentes méthodes d'apprentissage sur les graphes temporels. J'ai travaillé avec plusieurs architectures de prédiction de lien et étudié différentes manières de modifier leur apprentissage pour les adapter à une tâche de détection d'anomalies.

Une partie importante de ce travail a porté sur la génération d'exemples négatifs. En faisant varier la manière dont ces exemples sont construits, j'ai pu modifier les comportements que le modèle apprend. J'ai

notamment expérimenté des approches basées sur différentes heuristiques ainsi qu'une méthode utilisant un modèle de diffusion pour générer de nouvelles représentations.

Ces expérimentations m'ont permis de développer une démarche en formulant une hypothèse, en modifiant un élément du protocole d'apprentissage, puis en analysant les résultats. J'ai ainsi appris à chercher à comprendre les mécanismes à l'origine des résultats plutôt qu'à simplement comparer des scores.

9.2.3 Comparer différents modèles et paramètres

La comparaison des modèles a constitué une autre part importante du stage. Plusieurs architectures de prédiction de lien ont été étudiées et évaluées dans les mêmes conditions expérimentales. Cette comparaison doit distinguer les performances liées aux architectures de celles liées aux autres choix méthodologiques, comme la représentation des données ou la génération des exemples négatifs.

J'ai appris à choisir des tâches et des métriques adaptées à la problématique étudiée, notamment en faisant la distinction entre la prédiction de lien et la détection d'anomalies. De même, le fort déséquilibre entre les classes rend certaines métriques moins informatives que d'autres pour évaluer la détection d'attaques.

9.2.4 Communication scientifique

La présentation de mes travaux lors de deux workshops m'a permis de développer mes compétences en communication scientifique. J'ai notamment appris à sélectionner les informations essentielles de mes travaux et à les organiser pour présenter le problème étudié, la méthodologie et les premiers résultats. Ces présentations m'ont également appris à adapter mon discours à une audience qui ne connaît pas nécessairement les détails du projet. Le fait de réaliser ces présentations en anglais a été une expérience supplémentaire de communication dans un contexte international.

Plus généralement, ce stage m'a appris à communiquer régulièrement sur l'avancement d'un projet de recherche. Les échanges avec mes encadrants et les présentations m'ont amené à expliquer mes choix et à justifier mes hypothèses. Cette capacité à présenter son travail constitue une compétence importante que je souhaite continuer à développer.

9.3 Impact sur le projet professionnel

Ce stage constitue ma première expérience au sein d'un laboratoire de recherche. Cette immersion de plusieurs mois m'a permis de découvrir le fonctionnement d'une équipe de recherche et de mieux comprendre les différentes étapes d'un projet scientifique.

J'ai particulièrement apprécié la dimension méthodologique de la recherche. Il est nécessaire de formuler des hypothèses, de concevoir des expériences, d'analyser les résultats et de remettre en question les approches envisagées. Cela rend ce travail stimulant sur le plan intellectuel. J'ai également été sensible au fait que la recherche vise à produire de nouvelles connaissances, ce qui donne un sens particulier au travail réalisé.

Ce stage m'a également amené à réfléchir à la poursuite de mes études. Une poursuite en thèse était envisagée à l'issue du stage. Toutefois, malgré l'intérêt que je porte au milieu de la recherche, j'ai choisi de ne pas m'engager immédiatement dans une thèse de trois ans. Ce choix n'est pas un désintérêt pour la recherche, mais la volonté de prendre davantage de recul sur mon projet professionnel avant de m'investir dans un engagement de longue durée.

À court terme, je souhaite donc m'orienter vers un poste d'ingénieur de recherche ou un emploi proche du monde académique, me permettant de continuer à travailler sur des problématiques intéressantes tout en poursuivant ma réflexion sur une éventuelle thèse. Cette expérience a confirmé mon intérêt pour la

recherche et me laisse envisager la réalisation d'un doctorat dans le futur, lorsque mon projet professionnel sera davantage mûri.

10 Conclusion

Ce stage m'a permis de découvrir le fonctionnement de la recherche académique à travers un sujet exigeant. Les problématiques étudiées et la diversité des approches explorées en ont fait une expérience très enrichissante.

Au cours de ces cinq mois, j'ai acquis de nombreuses connaissances dans des domaines que je connaissais peu auparavant, notamment l'apprentissage sur les graphes, les graphes temporels, les modèles de langage ou encore les modèles de diffusion. Ce stage m'a également permis de mener un projet de recherche dans son ensemble, de l'étude bibliographique à l'implémentation, les expérimentations, l'analyse des résultats et leur présentation.

Au-delà des compétences techniques développées, cette expérience m'a permis de mieux comprendre le fonctionnement du métier de chercheur. Elle a confirmé mon intérêt pour cet environnement de travail.

À l'issue de ce stage, je souhaite poursuivre dans un environnement proche de la recherche afin de continuer à approfondir mes compétences en apprentissage statistique. Cette expérience constitue une étape importante dans la construction de mon projet professionnel et me permettra de faire un choix plus éclairé pour une éventuelle poursuite en thèse.

Références

- Cong, W., Zhang, S., Kang, J., Yuan, B., Wu, H., Zhou, X., Tong, H. et Mahdavi, M. (2023). “Do We Really Need Complicated Model Architectures For Temporal Networks?” In : *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. URL : <https://openreview.net/forum?id=ayPPc0SyLv1>.
- Devlin, J., Chang, M., Lee, K. et Toutanova, K. (2019). “BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding”. In : *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics : Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. Sous la dir. de J. Burstein, C. Doran et T. Solorio. Association for Computational Linguistics, p. 4171-4186. DOI : [10.18653/V1/N19-1423](https://doi.org/10.18653/V1/N19-1423).
- Dijoud, F., Gimenez, P.-F., Hurfin, M., Majorczyk, F. et Pilastre, B. (2026). “GRAAL : GRAPh-based Analysis of Logs for Advanced AI-based Intrusion Detection Systems”. In : *11th IEEE European Symposium on Security and Privacy (EuroSP 2026)*. IEEE European Symposium on Security et Privacy.
- Emma, K. (2025). “Base3 : a simple interpolation-based ensemble method for robust dynamic link prediction”. In : *CoRR abs/2506.12764*. DOI : [10.48550/ARXIV.2506.12764](https://doi.org/10.48550/ARXIV.2506.12764). arXiv : [2506.12764](https://arxiv.org/abs/2506.12764).
- Guan, W., Cao, J., Qian, S. et Gao, J. (2024). “LogLLM : Log-based Anomaly Detection Using Large Language Models”. In : *CoRR abs/2411.08561*. DOI : [10.48550/ARXIV.2411.08561](https://doi.org/10.48550/ARXIV.2411.08561). arXiv : [2411.08561](https://arxiv.org/abs/2411.08561).
- Lu, X., Sun, L., Zhu, T. et Lv, W. (2024). “Improving Temporal Link Prediction via Temporal Walk Matrix Projection”. In : *Advances in Neural Information Processing Systems 37 : Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*. Sous la dir. d’A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak et C. Zhang. URL : http://papers.nips.cc/paper%5C_files/paper/2024/hash/f7f7bf6014f7826da531aa50f4538ee19-Abstract-Conference.html.
- Mikolov, T., Chen, K., Corrado, G. et Dean, J. (2013). “Efficient Estimation of Word Representations in Vector Space”. In : *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*. Sous la dir. d’Y. Bengio et Y. LeCun. URL : <http://arxiv.org/abs/1301.3781>.
- Nguyen, T.-K. et Fang, Y. (2024). “Diffusion-based Negative Sampling on Graphs for Link Prediction”. In : *Proceedings of the ACM Web Conference 2024. WWW ’24*. Singapore, Singapore : Association for Computing Machinery, p. 948-958. ISBN : 9798400701719. DOI : [10.1145/3589334.3645650](https://doi.org/10.1145/3589334.3645650).
- Reha, J., Lovisotto, G., Russo, M., Gravina, A. et Grohnfeldt, C. (2023). “Anomaly Detection in Continuous-Time Temporal Provenance Graphs”. In : *Temporal Graph Learning Workshop @ NeurIPS 2023*. URL : <https://openreview.net/forum?id=88tG1xxhsf>.
- Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F. et Bronstein, M. (2020). “Temporal Graph Networks for Deep Learning on Dynamic Graphs”. In : *ICML 2020 Workshop on Graph Representation Learning*.
- Xu, D., Ruan, C., Körpeoglu, E., Kumar, S. et Achan, K. (2020). “Inductive representation learning on temporal graphs”. In : *8th International Conference on Learning Representations, ICLR 2020, Addis*

Ababa, Ethiopia, April 26-30, 2020. OpenReview.net. URL : <https://openreview.net/forum?id=rJeWlyHYwH>.

Yu, L., Sun, L., Du, B. et Lv, W. (2023). “Towards Better Dynamic Graph Learning : New Architecture and Unified Library”. In : *Advances in Neural Information Processing Systems 36 : Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Sous la dir. d’A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt et S. Levine. URL : http://papers.nips.cc/paper%5C_files/paper/2023/hash/d611019afba70d547bd595e8a4158f55-Abstract-Conference.html.

Zhang, X., Wang, Y., Wang, X. et Zhang, M. (2024). “Efficient Neural Common Neighbor for Temporal Graph Link Prediction”. In : *CoRR* abs/2406.07926. DOI : [10.48550/ARXIV.2406.07926](https://doi.org/10.48550/ARXIV.2406.07926). arXiv : [2406.07926](https://arxiv.org/abs/2406.07926).

A Annexes

TABLE 4 – Performances des différents encodages en prédiction de lien et en détection d'anomalies.

Encodage	Modèle	Prédiction de liens			Détection d'anomalies		
		Acc.	AUPRC	AUROC	F1	AUPRC	AUROC
Sans features	GraphMixer	0.7954	0.9675	0.9592	0.0033	0.0011	0.6480
	DyGFormer	0.8257	0.9642	0.9556	0.0041	0.0012	0.6415
	TGAT	0.7552	0.9644	0.9518	0.0035	0.0008	0.4171
Word2Vec	GraphMixer	0.7954	0.9675	0.9594	0.0031	0.0010	0.6474
	DyGFormer	<u>0.8747</u>	0.9629	0.9539	0.0034	0.0011	0.6081
	TGAT	0.8086	0.9592	0.9471	0.0024	0.0008	0.5202
BERT pré-entraîné	GraphMixer	0.7761	<u>0.9908</u>	<u>0.9903</u>	<u>0.0061</u>	0.0017	0.7304
	DyGFormer	0.8899	0.9854	0.9837	0.0051	0.0013	0.6916
	TGAT	0.8378	0.9751	0.9706	0.0073	0.0012	0.6284
BERT fine-tuné	GraphMixer	0.8615	0.9916	0.9911	0.0052	0.0015	0.7025
	DyGFormer	0.8646	0.9684	0.9665	0.0060	0.0013	0.6645
	TGAT	0.8502	0.9765	0.9725	0.0047	0.0013	0.7060
BERT + type	GraphMixer	0.8532	0.9873	0.9871	0.0043	<u>0.0016</u>	<u>0.7181</u>
	DyGFormer	0.8086	0.9736	0.9686	0.0029	0.0011	0.6268
	TGAT	0.8660	0.9895	0.9887	0.0057	0.0008	0.4836

TABLE 5 – Performances des différentes stratégies de génération d'exemples négatifs en prédiction de lien et en détection d'anomalies.

Encodage	Modèle	Prédiction de liens			Détection d'anomalies		
		Acc.	AUPRC	AUROC	F1	AUPRC	AUROC
Aléatoire	GraphMixer	0.8532	0.9873	0.9871	0.0043	0.0016	0.7181
	DyGFormer	0.8660	0.9895	0.9887	0.0029	0.0011	0.6268
	TGAT	0.8086	0.9736	0.9686	0.0057	0.0008	0.4836
Historique	GraphMixer	0.8997	0.9842	0.9853	0.0058	0.0014	0.5652
	DyGFormer	0.7089	0.9559	0.9831	0.0030	0.0010	0.5563
	TGAT	0.9039	0.9856	0.9831	0.0054	0.001	0.5171
Popularité	GraphMixer	0.8972	0.9853	0.9754	0.0022	0.0005	0.3131
	DyGFormer	0.9438	0.9817	0.9836	0.0023	0.0005	0.2920
	TGAT	0.8999	0.9657	0.9684	0.0025	0.0005	0.2693
Voisins de voisins	GraphMixer	0.9580	0.9923	0.9932	0.0059	0.0021	0.7628
	DyGFormer	0.9616	0.9948	0.9949	0.0063	0.0020	0.7902
	TGAT	0.9475	0.9817	0.9894	0.0099	0.0019	0.7622
Motif de taille 2	GraphMixer	0.9642	0.9963	0.9962	0.0063	0.0016	0.7000
	DyGFormer	0.9398	0.9909	0.9901	0.0132	0.0017	0.6578
	TGAT	0.8114	0.9734	0.9675	0.0022	0.0009	0.5939
Motif global	GraphMixer	0.9959	0.9997	0.9996	<u>0.0112</u>	0.0035	0.8538
	DyGFormer	0.9957	0.9996	0.9994	0.0102	<u>0.0027</u>	<u>0.8353</u>
	TGAT	0.9959	<u>0.9996</u>	<u>0.9995</u>	0.0078	0.0023	<u>0.8027</u>

